THIRD YEAR COMPUTER ENGINEERING — ARTIFICIAL INTELLIGENCE LAB

Development of a Simple Chatbot Using Predefined Rules and Pattern Matching

Academic Practical Manual & Write-up

1. TITLE & AIM

Title: Development of a Simple Chatbot Using Predefined Rules and Pattern Matching.

Aim: To design and implement a rule-based chatbot using Python, regular expressions, and pattern-matching techniques to simulate interactive user conversations.

2. THEORY

A rule-based chatbot operates on deterministic logic. It analyzes user inputs using pattern matching (typically regular expressions) and maps identified patterns to appropriate, preconfigured response templates.

Key concepts involved include:

- **Pattern Matching:** Identifying key words, phrases, or sentence structures in user input using regular expressions (`re` module in Python).
- **Intent Identification:** Categorizing the user's input (e.g., greetings, queries, farewells) based on matched patterns.
- **Predefined Responses:** Returning static or semi-dynamic answers mapped directly to the identified intent.

3. REQUIREMENTS

Component	Specification / Tools
Hardware	Standard PC / Laptop (Intel i3/AMD Ryzen 3 or higher, 4 GB RAM minimum)
Software	Python 3.x Environment
IDE / Tools	VS Code, Jupyter Notebook, PyCharm, or standard Python IDLE

4. PROCEDURE

1. Import the necessary standard Python libraries (`re` and `random`).
2. Define a structured data layout (such as list of tuples) containing regular expression patterns paired with response options.

3. Construct a function to take user text, normalize it (e.g., lowercasing and trimming), and iteratively evaluate it against defined patterns.
4. If a match occurs, return a randomly selected response from the corresponding tuple.
5. Implement a default fallback response when no pattern matches the user input.
6. Create an interactive command-line loop allowing continuous conversation until an exit rule is triggered.

5. IMPLEMENTATION CODE

The source code below is styled with blue line highlights and custom formatting:

```
import re
import random

# Rules definition: (regex pattern, tuple of possible responses)
RULES = [
    (r'hi|hello|hey|greetings',
     ("Hello! How can I assist you today?", "Hi there! What can I help you with?")),

    (r'what is your name\??',
     ("I am RuleBot, a simple pattern-matching chatbot.", "You can call me RuleBot!")),

    (r'how are you\??',
     ("I am just a computer program, but I am operating at peak efficiency!", "I'm doing
well, thanks for asking!")),

    (r'what can you do\??',
     ("I can answer basic questions using predefined rules and regex patterns.", "I
assist with simple pattern-matched queries.")),

    (r'(.*) (location|located)\??',
     ("I exist inside your computer terminal!", "I live in the digital realm.")),

    (r'bye|quit|exit|goodbye',
     ("Goodbye! Have a great day ahead.", "Bye! Feel free to reach out anytime.")),
]

def get_response(user_input):
    cleaned_input = user_input.strip().lower()

    # Check user input against defined rules
    for pattern, responses in RULES:
        if re.search(pattern, cleaned_input):
            return random.choice(responses), pattern == r'bye|quit|exit|goodbye'

    # Fallback response if no pattern matches
    fallback_responses = (
        "I'm sorry, I don't quite understand that. Could you rephrase?",
        "I am limited to simple rules. Try asking about my name or capabilities."
    )
    return random.choice(fallback_responses), False

def start_chatbot():
    print("--------------------------------------------------")
    print("RuleBot initialized. Type 'bye' or 'exit' to quit.")
```

```
    print("--------------------------------------------------")

    while True:
        user_input = input("You: ")
        if not user_input.strip():
            continue

        response, should_exit = get_response(user_input)
        print(f"Bot: {response}")

        if should_exit:
            break

if __name__ == "__main__":
    start_chatbot()
```

6. SAMPLE OUTPUT

```
--------------------------------------------------
RuleBot initialized. Type 'bye' or 'exit' to quit.
--------------------------------------------------
You: Hello
Bot: Hello! How can I assist you today?
You: What is your name?
Bot: I am RuleBot, a simple pattern-matching chatbot.
You: Where are you located?
Bot: I exist inside your computer terminal!
You: Tell me a joke
Bot: I'm sorry, I don't quite understand that. Could you rephrase?
You: bye
Bot: Goodbye! Have a great day ahead.
```

7. ANALYSIS AND EVALUATION

- **Time Complexity:** $O(N \times M)$ per response, where N is the number of defined rules and M is the average string length evaluated by the regular expression engine.
- **Space Complexity:** $O(R)$, where R represents stored pattern rules and response sets in memory.
- **Strengths:** Predictable and reliable execution for expected inputs, low latency, lightweight footprint, zero dependence on external APIs or heavy models.
- **Limitations:** Inability to handle semantic variations outside hardcoded patterns, context tracking limitations, and sensitivity to complex syntax/typos.

8. CONCLUSION

A simple rule-based chatbot was successfully implemented using pattern matching and regular expressions in Python. The practical highlights how rule-driven conditional matching handles task-specific dialog flows while

illustrating the fundamental differences between static matching algorithms and probabilistic machine learning systems.

9. VIVA QUESTIONS & ANSWERS

Q1: What is the difference between a rule-based chatbot and a self-learning (AI/ML) chatbot?
Answer: Rule-based chatbots rely on hardcoded patterns (regex/if-else logic) to deliver predefined static answers. Self-learning chatbots utilize Natural Language Processing (NLP) and Machine Learning models to derive context, intent, and generate probabilistic responses dynamically.

Q2: What role do regular expressions play in pattern matching?
Answer: Regular expressions allow the chatbot to search for specific character sequences, keywords, or structural variations within user inputs, enabling flexible intent recognition beyond rigid string equality checks.

Q3: What is a fallback mechanism in chatbots?
Answer: A fallback mechanism provides a graceful default answer when a user input fails to match any existing pattern or rule, preventing application errors or silent failures.

Q4: How can you improve a rule-based chatbot's handling of varied sentence structures?
Answer: Preprocessing steps such as text lowercasing, stemming/lemmatization, stop-word removal, and expanding regex dictionary definitions help improve structural adaptability.